# ReviewPilot - AI PR Review & Test Planner

Assignment: AI Specialist Engineer - DriveNets

Candidate: Ilay Romi

Date: May 2026

Repository: Not included in this submission

Recorded demo: Not included; local demo commands are provided below

## 1. Executive Summary

ReviewPilot is a command-line tool I built with Claude Code to support pull-request review. Given a local unified .diff file, it produces a structured Markdown report that helps a reviewer understand what changed, where the risk is, and what should be tested.

The project goal was not to build a naive LLM wrapper. I intentionally separated deterministic engineering logic from AI-assisted suggestions:

- Deterministic layer: parse the diff, classify files, compute transparent risk signals, and build a structured report context.
- AI-assisted layer: generate regression hypotheses, test suggestions, and a reviewer checklist from that structured context.

The most important design decision is that the AI client never receives raw diff text. It receives only an AnalysisContext: file summaries, roles, risk scores, and detected signal labels. This makes the core logic testable, the AI input inspectable, and the output easier to evaluate.

Outcome: the MVP is working end-to-end. It includes a CLI (python -m reviewpilot), four realistic sample diffs, a generated sample report, full README and development log, and 451 passing tests. The current AI layer uses MockAIClient intentionally: it is deterministic, offline, and satisfies the same AIClient protocol that a real Anthropic client could implement later.

## 2. Problem and Approach

Code reviewers often receive a multi-file diff without a clear signal about where to focus. They need to answer four questions quickly: what changed, which files are risky, what tests are missing, and what should be verified manually.

Sending a raw diff directly to an LLM is not sufficient. It is difficult to test, can produce non-deterministic results, and may hallucinate behavior not present in the diff. ReviewPilot addresses this by first producing a deterministic analysis, then using AI only for clearly labeled suggestions.

The generated report separates computed facts from hypotheses. Risk levels come from explicit rules, while AI-assisted sections are framed as suggestions for human validation.

## 3. Architecture

```
.diff file
  -> parser.py
  -> classifier.py
  -> risk_scorer.py
  -> AnalysisContext
  -> MockAIClient
  -> ReviewReport
  -> Markdown renderer / CLI output
```

| Component | Responsibility | Verification |
|---|---|---|
| models.py | Typed dataclasses and enums used across the pipeline | Model construction and property tests |
| parser.py | Parses unified diff text into structured diff objects | Unit tests for change types and invalid diffs |
| classifier.py | Maps paths to roles such as SOURCE, TEST, MIGRATION, DOCS | Unit tests for patterns, edge cases, and precedence |
| risk_scorer.py | Computes deterministic risk signals and risk levels | Unit tests for every rule and boundary |
| ai_client.py | Defines AIClient protocol and deterministic MockAIClient | Grounding, determinism, and protocol tests |
| report_builder.py | Orchestrates parse -> classify -> score -> context -> AI insights | Integration tests |
| renderer.py | Converts ReviewReport to Markdown | Renderer tests for sections, tables, and escaping |
| cli.py | User-facing command-line interface | CLI tests for stdout, file output, and errors |

## Deterministic vs AI-assisted design

| Layer | Produces | Why it matters |
|---|---|---|
| Deterministic | File roles, risk signals, scores, report structure | Fully testable and explainable |
| AI-assisted | Regression hypotheses, test suggestions, reviewer checklist | Useful for reviewer thinking, but labeled as suggestions |

The AnalysisContext boundary gives four benefits: reproducible prompts, inspectable AI input, reduced exposure of raw code, and a clean path to replacing MockAIClient with a real Anthropic client later.

## 4. Claude Code Development Process

I used Claude Code as an engineering assistant, not as an unchecked code generator. The workflow was: plan first, implement one module per commit, require tests with each module, verify before committing, and use git checkpoints.

| Stage | Result |
|---|---|
| Scaffold | Project structure, pyproject.toml, smoke test |
| Models | Dataclasses and enums |
| Parser | Unified diff parser |
| Classifier | Path-based file role classifier |
| Risk scorer | Deterministic weighted risk rules |
| AI client | AIClient protocol and MockAIClient |
| Report builder | Full pipeline orchestration |
| Renderer | Markdown report generation |
| CLI | User-facing command line tool |
| Documentation | README, dev log, sample diffs, sample report, final submission report |

### Prompting patterns used

- Planning prompt: define architecture, data models, risks, and implementation order without coding.
- Module prompt: implement one file only, with explicit constraints and no unrelated changes.
- Testing prompt: create focused tests for specific behavior and edge cases.
- Failure-analysis prompt: explain the root cause before changing code.
- Documentation prompt: write for a technical reviewer, with accurate limitations and no exaggerated claims.

This made the AI output easier to review and kept the project from turning into a broad, untestable demo.

## 5. Testing Strategy

The assignment asked for a short explanation of what was tested and how the tests fit together. I used four layers: unit tests per module, integration tests for report_builder and renderer, CLI tests for user-facing behavior, and manual checks on realistic sample diffs.

All AI-related tests use MockAIClient. This avoids network calls, API keys, non-determinism, and token cost while still testing the AI boundary and report flow.

| Milestone | Tests passing |
|---|---|
| Parser | 89 |
| Classifier | 178 |
| Risk scorer | 244 |
| AI client | 290 |
| Report builder | 350 |
| Renderer | 415 |
| Final CLI/project | 451 |

I did not implement golden-output evaluation for a real LLM because the current MVP does not call a live model. A golden evaluation set is listed as future work for the real API version.

## 6. Example of AI Output I Evaluated and Corrected

The clearest issue caught during development was in the parser tests. Claude generated a test fixture where one test expected three context lines in a unified diff, but the parser returned two. Instead of changing the parser immediately, I investigated whether the failure was caused by the parser or by the test fixture.

The root cause was the fixture: in unified diff format, a blank context line must be represented as a line starting with one space (" "), not as a bare empty line (""). The parser was correct; the test fixture was malformed.

Resolution: I fixed the fixture and kept the parser behavior unchanged. Weakening the parser would have made the test pass while making the implementation less correct.

Other verification issues were handled the same way: I created a local virtual environment when pytest was missing globally, used pytest instead of long shell spot-checks, and caught an initially untracked __main__.py file with git status before finalizing the CLI commit.

## 7. Critical Reflection

Claude was very effective for structure, boilerplate, tests, and documentation once the task was well constrained. It accelerated implementation, but it did not replace engineering judgment.

| Claude was strong at | I retained ownership of |
|---|---|
| Generating consistent module structure and docstrings | Defining and preserving the deterministic/AI boundary |
| Creating broad initial test coverage from explicit requirements | Limiting scope to a realistic MVP |
| Helping diagnose failures when asked to reason first | Deciding not to add a real API client yet |
| Drafting documentation for a stated audience and tone | Reviewing tests for meaningful assertions and commit hygiene |

The main lesson is that Claude Code is strongest when directed with clear constraints. The developer still owns the architecture, verification strategy, and final quality bar.

## 8. What I Chose Not To Test

| Not tested | Why acceptable for MVP |
|---|---|
| Real LLM output quality | Current version uses MockAIClient; real model evaluation needs a golden-output suite |
| Very large diffs | MVP targets typical PR diffs, not multi-megabyte changes |
| Full malformed diff recovery | Parser supports standard git diff output and clear error cases |
| Runtime behavior of changed code | ReviewPilot analyzes diffs; it does not execute the target application |
| Semantic code understanding | Risk scoring is heuristic and keyword-based by design |
| GitHub/GitLab integration | Local .diff files were the intended scope |

The accepted risk is that the tool provides review guidance, not proof of correctness. It helps reviewers focus, but it does not replace tests, code ownership, or manual review.

## 9. Limitations and Future Work

### Current limitations

- Local .diff files only.
- MockAIClient only; no live LLM API in the current MVP.
- Heuristic risk scoring; no AST or control-flow analysis.
- No repository history, dependency graph, or coverage data.
- Markdown output only.

### Future improvements

- Add a real AnthropicAIClient behind the existing AIClient protocol.
- Add GitHub PR integration and PR comments.
- Make risk rules configurable through reviewpilot.toml.
- Enrich AnalysisContext with repository history and test coverage.
- Add a golden-output evaluation suite for real LLM responses.
- Add CI integration and richer output formats such as HTML or SARIF.

## 10. Demo

Run the project locally:

```
python -m venv .venv
.\.venv\Scripts\activate          # Windows
pip install -e ".[dev]"
```

Generate a sample report:

```
python -m reviewpilot examples/auth_change.diff --output reports/sample_report.md
```

Run all tests:

```
python -m pytest tests/ -v
# Expected result: 451 passed
```

Sample inputs are under examples/; the generated report is committed at reports/sample_report.md.

Repository link: Not included in this submission. Recorded demo: Not included; local demo commands are provided above.

## 11. Conclusion

ReviewPilot demonstrates how I approach AI-assisted engineering: start from a clear problem, define strict boundaries, use Claude Code to accelerate implementation, and verify every output before accepting it.

The project is intentionally modest in scope, but complete as an MVP: it has a working CLI, realistic examples, deterministic risk analysis, AI-assisted suggestions through a clean protocol boundary, documentation, and 451 passing tests.

Claude was useful as a collaborator for constrained tasks. The architecture, scope, testing strategy, and final quality decisions remained mine.

ReviewPilot - developed as the AI Specialist Engineer assignment for DriveNets, May 2026.